

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ»
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет «Школа информатики, физики и технологий»

Кураленок Святослав Игоревич

Отказоустойчивый инференс Large Scale MoE моделей

Выпускная квалификационная работа – БАКАЛАВРСКАЯ РАБОТА
по направлению подготовки
01.03.02 Прикладная математика и информатика
образовательная программа «Прикладная математика и информатика»

Научный руководитель:
доцент департамента информатики
Овчинников Андрей Анатольевич

Соруководитель:
к. ф.-м. н., руководитель подразделения
«AI Studio Tech»
ООО «Яндекс.Технологии»
Ершов Василий Алексеевич

Санкт-Петербург
2026

Содержание

Аннотация	3
Abstract	4
Используемые определения и термины	5
Введение	8
Цели и задачи	9
Достигнутые результаты	10
Структура работы	11
Глава 1 Обзор литературы	12
1.1 Большие языковые модели и инференс	12
1.2 Архитектура Mixture-of-Experts	13
1.3 Методы распределенного инференса	14
1.4 Коллективные коммуникации и ограничения распределенного инференса	15
1.5 ДеерЕР и коллективы для МоЕ	16
1.6 Движки адресной передачи памяти: NIXL и MoonCake Transfer Engine	17
Выводы по главе	18
Глава 2 Метод	20
2.1 Описание метода	20
2.2 Архитектура метода	21
2.3 Компонент DistMoEBlock	23
2.4 Компонент DistExpertsBlock	24
2.5 Компонент TensorTransferEngine	25
Выводы по главе	27
Глава 3 Эксперименты	28
3.1 Методика экспериментальной оценки	28
3.2 Производительность МоЕ инференса	28
3.3 Балансировка нагрузки и отказоустойчивость	30
Выводы по главе	31
Заключение	33
Список использованных источников	36

Аннотация

Для практического применения систем искусственного интеллекта недостаточно обучить модель: необходимо организовать ее вывод, то есть выполнение модели на пользовательских запросах с заданными требованиями к задержке, пропускной способности и доступности. Такой этап работы модели принято называть инференсом. Современный распределенный инференс больших языковых моделей, основанных на архитектуре Mixture-of-Experts (MoE), представляет собой комплексную системную задачу. Многие существующие решения используют коллективные коммуникации и в первую очередь оптимизируют пропускную способность при фиксированном составе участников [1; 2]. Однако такая схема плохо подходит для динамической балансировки нагрузки между экспертными блоками: перераспределение запросов, использование реплик экспертов и добавление/исключение исполнителей на лету требуют более гибкой модели взаимодействия.

В рамках исследования предложена альтернативная клиент-серверная архитектура взаимодействия между основной частью модели и экспертными блоками. Разработан прототип системы, поддерживающий асинхронную диспетчеризацию запросов, мониторинг состояния экспертных блоков, динамический выбор доступного экспертного блока и обработку частичных отказов в процессе инференса. Для уменьшения накладных расходов, возникающих при замене коллективных операций адресным взаимодействием с экспертными блоками, реализован ряд оптимизаций, направленных на повышение эффективности передачи данных и вычислений на GPU.

Проведенные эксперименты показали, что предложенный подход обеспечивает возможность динамической балансировки нагрузки между экспертными блоками и сохранение доступности системы при частичных отказах за счет выбора доступных реплик экспертов. Полученные результаты подтверждают перспективность использования клиент-серверного подхода для построения гибких распределенных систем инференса MoE моделей [2; 3].

Данная работа состоит из 36 страниц, 3 глав, 11 рисунков, 1 таблицы. Использовано 30 источников.

Ключевые слова: большие языковые модели; инференс; Mixture-of-Experts; MoE; клиент-серверная архитектура; отказоустойчивость; балансировка нагрузки; распределенные вычисления; GPU.

Abstract

Practical use of artificial intelligence systems requires more than training a model: the system must also execute the model on user requests under latency, throughput, and availability constraints. This stage of model operation is commonly referred to as inference. Modern distributed inference of large language models based on the Mixture-of-Experts (MoE) architecture is a complex systems problem. Many existing solutions rely on collective communication and primarily optimize throughput for a fixed set of participants [1; 2]. However, this scheme is poorly suited for dynamic load balancing across expert blocks: request redistribution, use of expert replicas, and adding or excluding workers at runtime require a more flexible interaction model.

This research proposes an alternative client-server architecture for interaction between the main model and expert blocks. A prototype system was developed that supports asynchronous request dispatching, expert block state monitoring, dynamic selection of available expert blocks, and runtime handling of partial failures. To reduce the overhead introduced by replacing collective operations with direct interaction with expert blocks, several optimizations aimed at improving GPU computation and data transfer efficiency were implemented.

The experiments show that the proposed approach enables dynamic load balancing across expert blocks and preserves system availability under partial failures by selecting available expert replicas. The obtained results confirm the potential of the client-server approach for building flexible distributed inference systems for MoE models [2; 3].

The paper contains 36 pages, 3 chapters, 11 figures, 1 table. 30 sources are used.

Keywords: large language models; inference; Mixture-of-Experts; MoE; client-server architecture; fault tolerance; load balancing; distributed computing; GPU.

Используемые определения и термины

All-to-All – коллективная операция обмена данными, при которой каждый участник вычисления передает данные всем остальным участникам и получает данные от них.

AllGather – коллективная операция, при которой каждый участник передает свой фрагмент данных всем остальным участникам, после чего у каждого участника оказывается полный набор фрагментов.

AllReduce – коллективная операция, при которой значения от всех участников объединяются с помощью заданной операции, например суммирования, и результат становится доступен всем участникам.

Combine – этап MoE слоя, на котором результаты вычислений выбранных экспертов собираются обратно и приводятся к порядку исходных входных активаций.

DeepEP – специализированная библиотека для высокопроизводительных коммуникаций в MoE моделях, оптимизирующая операции `dispatch` и `combine` при экспертном параллелизме.

Dispatch – этап MoE слоя, на котором входные активации или токены отправляются к экспертам, выбранным роутером.

Expert parallelism (EP) – способ распределенного выполнения MoE моделей, при котором разные эксперты размещаются на разных вычислительных устройствах или узлах.

Grouped GeMM (General Matrix Multiplication) – выполнение нескольких операций матричного умножения как одной сгруппированной операции для уменьшения накладных расходов запуска и более эффективного использования GPU.

KV кэш – набор сохраненных ключей и значений механизма внимания, переиспользуемый на последующих шагах авторегрессионного инференса.

Mixture-of-Experts (MoE) – архитектура нейронной сети, в которой часть полностью связанных слоев заменяется набором специализированных экспертных блоков, а роутер выбирает подмножество экспертов для обработки каждого входа.

MoonCake Transfer Engine – библиотека для высокопроизводительной передачи данных между вычислительными узлами, используемая в работе как одна из реализаций интерфейса передачи тензоров.

NCCL (NVIDIA Collective Communication Library) – библиотека NVIDIA для коллективных коммуникаций и адресных передач между GPU, широко используемая в распределенном обучении и инференсе нейронных сетей.

NIXL (NVIDIA Inference Xfer Library) – библиотека для низкоуровневой высокопроизводительной передачи данных, используемая в работе как одна из реализаций интерфейса передачи тензоров.

NVSHMEM (NVIDIA OpenSHMEM) – библиотека, реализующая модель OpenSHMEM для кластеров GPU и предоставляющая односторонний доступ к памяти, синхронизацию и коллективные операции.

Pipeline parallelism (PP) – метод распределенного выполнения модели, при котором последовательные части модели размещаются на разных устройствах и выполняются как стадии конвейера.

RDMA (Remote Direct Memory Access) – механизм прямого доступа к памяти удаленного узла, позволяющий передавать данные с минимальным участием центрального процессора.

ReduceScatter – коллективная операция, при которой данные от всех участников сначала объединяются, а затем результат распределяется между участниками по фрагментам.

Tensor parallelism (TP) – метод распределенного выполнения модели, при котором отдельные тензорные операции, например матричные умножения, разбиваются между несколькими устройствами.

Top-k маршрутизация – способ маршрутизации в MoE слое, при котором для каждого входа выбираются k наиболее подходящих экспертов по оценкам роутера.

Авторегрессионный инференс – режим вывода, при котором модель последовательно предсказывает следующий токен на основе уже имеющегося контекста и ранее полученных токенов.

Адресная передача – передача данных между конкретными участниками распределенной системы без обязательного участия всей коммуникационной группы.

Асинхронная диспетчеризация – способ обработки запросов, при котором отправка задач экспертам, ожидание результатов и продолжение работы системы не блокируются одной общей точкой выполнения.

Балансировка нагрузки – распределение запросов или вычислительных задач между доступными экспертными блоками для более равномерной загрузки ресурсов.

Большая языковая модель – нейронная модель, обученная на больших объемах текстовых данных и предназначенная для решения задач обработки естественного языка.

Вычислительный узел – сервер или отдельный процесс-исполнитель, участвующий в распределенном выполнении модели и располагающий вычислительными ресурсами, например GPU.

Коллективная операция – коммуникационная операция, в которой согласованно участвует группа процессов или устройств.

Коммуникационная группа – набор участников распределенного вычисления, между которыми выполняются коллективные операции или другие согласованные коммуникации.

Маршрутизация токенов – процесс выбора экспертов, которым должны быть отправлены токены или соответствующие им активации в МоЕ слое.

Отказоустойчивость – способность системы сохранять работоспособность или деградировать контролируемым образом при отказе части вычислительных узлов.

Пропускная способность – количество запросов, токенов или байтов данных, которое система способна обработать или передать за единицу времени.

Ранг – отдельный участник распределенного вычисления, обычно соответствующий процессу, GPU или их связке в рамках коммуникационной группы.

Роутер – компонент МоЕ слоя, определяющий, какие эксперты должны обработать конкретные входные активации.

Трансформер – архитектура нейронной сети, основанная на механизме внимания и широко используемая в современных языковых моделях.

Эксперт – отдельный вычислительный блок внутри МоЕ слоя, обычно реализованный как полносвязная нейронная сеть и активируемый только для части входных данных.

Экспертное вычисление – этап МоЕ слоя, на котором выбранные эксперты обрабатывают переданные им активации.

Экспертный блок – логически выделенный компонент распределенной МоЕ системы, который содержит один или несколько экспертов, принимает активации от основной модели, выполняет экспертные вычисления и возвращает результаты; в данной работе экспертный блок также является единицей диспетчеризации, мониторинга и балансировки нагрузки.

Экспертный слой – слой МоЕ модели, содержащий роутер, набор экспертов и операции отправки активаций к экспертам и сборки результатов.

Введение

Практическое применение большой языковой модели связано с этапом вывода: модель получает пользовательский запрос, выполняет прямые проходы с уже зафиксированными параметрами и возвращает результат. Такой этап работы модели принято называть инференсом. Зачастую повышение качества больших языковых моделей на широком наборе задач достигается за счёт увеличения количества их параметров и вычислительного бюджета [4; 5]. Однако в традиционных плотных архитектурах трансформеров все параметры соответствующего слоя участвуют в вычислениях на каждом проходе модели, поэтому масштабирование таких моделей приводит к быстрому росту стоимости инференса [6; 7].

Эта проблема важна не только для отдельных моделей. Современные SOTA модели уже достигают масштаба в сотни миллиардов параметров: DeepSeek-V3.2 содержит 671 миллиардов параметров [8], а Switch Transformer и GLaM демонстрируют масштабирование MoE архитектур до триллионов параметров [9; 10]. Для таких моделей веса, KV кэш и промежуточные активации становятся слишком большими для одного GPU. Поэтому распределенный инференс является необходимым условием практического применения крупных моделей [1].

Одним из методов распределенного инференса является конвейерный параллелизм (pipeline parallelism, PP), при котором модель разбивается на последовательные части, размещаемые на разных GPU. Однако данный подход имеет ограничения для инференса больших языковых моделей из-за последовательного характера выполнения вычислений между стадиями конвейера, что приводит к увеличению времени выполнения отдельных запросов и неполной утилизации вычислительных ресурсов. Другим широко используемым подходом является тензорный параллелизм (tensor parallelism, TP), при котором вычисления внутри отдельных операций, например матричных умножений, распределяются между несколькими устройствами [11; 12]. В отличие от PP, TP позволяет сохранять низкое время выполнения для отдельных запросов и более эффективно загружать GPU, однако современные работы по тензорному параллелизму при инференсе отмечают, что по мере роста больших языковых моделей и распределения модели на большее число устройств коммуникационная стоимость увеличивается и может становиться узким местом производительности [13]. В результате даже сочетание различных методов параллелизма не позволяет масштабировать плотные архитектуры без существенного роста стоимости инференса.

Поэтому была предложена альтернативная архитектура: Mixture-of-Experts (MoE) [3]. Суть данного подхода заключается в том, что в блоке трансформера полносвязный линейный слой (feed-forward network) заменяется на множество меньших полносвязных блоков (экспертов), а также добавляется роутер, который определяет, какие из экспертов будут активны на текущем прямом проходе.

Ключевое преимущество MoE заключается не в том, что на каждом проходе ис-

пользуется фиксированная доля всех весов модели, а в том, что активируется только часть экспертных параметров. Плотные части модели продолжают выполняться полностью, однако вычисления в экспертных слоях становятся разреженными. За счет этого МоЕ позволяет увеличивать общую емкость модели без пропорционального роста стоимости одного прохода [9; 10].

Механизм масштабирования МоЕ связан с экспертным параллелизмом (expert parallelism, EP): разные эксперты размещаются на разных видеокартах, а токены после роутинга отправляются к выбранным экспертам. В классической схеме для этого используется примитив коллективных коммуникаций All-to-All: сначала активации передаются от одного ранга ко всем остальным, и так для каждого, затем результаты экспертных вычислений собираются обратно [2; 14]. Такие коллективные операции на NVIDIA GPU обычно реализуются поверх специализированных коммуникационных библиотек, в частности NCCL и NVSHMEM [15; 16].

Несмотря на высокую производительность, коллективная схема плохо подходит для динамической балансировки нагрузки. Роутер МоЕ выбирает экспертов в зависимости от потока входных данных, поэтому часть экспертов может регулярно активироваться на гораздо большем количестве токенов, чем остальные. На коллективах раскладка экспертов по рангам фиксируется заранее, а каждый шаг All-to-All связывает всех участников. Поэтому система не может во время работы свободно перенаправлять запросы к менее загруженной реплике эксперта, исключать перегруженный экспертный блок или добавлять новый без полного перезапуска всей системы.

Таким образом, возникает вопрос о том, можно ли заменить статичную коллективную схему взаимодействия в экспертном слое на архитектуру, позволяющую принимать решения о выборе экспертного блока во время работы системы.

Цели и задачи

Основной целью данной работы является предложить и разработать рабочий прототип альтернативного подхода к инференсу больших языковых моделей, основанных на архитектуре МоЕ, удовлетворяющий требованиям динамической балансировки нагрузки и обеспечивающий более высокую отказоустойчивость по сравнению с оригинальным подходом, основанным на коллективах.

Для достижения поставленной цели была выдвинута следующая гипотеза: замена коллективных All-to-All коммуникаций на уровне экспертного слоя в МоЕ моделях на клиент-серверное взаимодействие между основной частью модели и экспертными блоками позволяет выбирать конкретного исполнителя во время работы системы, перераспределять нагрузку между репликами экспертов и сохранять доступность при отказе части экспертных блоков.

Для проверки данной гипотезы ставятся следующие задачи:

- Разработать интерфейс для высокопроизводительной передачи данных между GPU,

а также две реализации этого интерфейса: для NIXL (NVIDIA Inference Xfer Library) и MoonCake Transfer Engine

- Разработать клиент-серверную архитектуру для распределенного инференса MoE моделей с поддержкой асинхронного взаимодействия между основной частью модели и экспертными блоками
- Разработать систему асинхронной диспетчеризации запросов с поддержкой очередей
- Разработать механизмы динамической балансировки нагрузки между экспертными блоками, включая выбор менее загруженной реплики эксперта
- Реализовать систему мониторинга состояния экспертных блоков и механизмы обнаружения отказов в режиме реального времени
- Провести оптимизацию сетевого взаимодействия для минимизации накладных расходов при передаче активаций и результатов вычислений
- Провести оптимизацию GPU-вычислений для экспертных блоков
- Провести экспериментальное сравнение производительности предложенного подхода с существующими коллективными решениями
- Проверить возможность динамической балансировки нагрузки и сохранения доступности при частичных отказах

Достигнутые результаты

В рамках данной работы был предложен и реализован прототип отказоустойчивого инференса MoE моделей, основанный на клиент-серверной архитектуре, подтверждающий выдвинутую гипотезу.

Для этого была разработана программный модуль, включающий следующие компоненты:

- Интерфейс для высокопроизводительных коммуникаций
- Асинхронная клиент-серверная архитектура, предоставляющая возможность динамического масштабирования экспертных блоков
- Система мониторинга и обнаружения отказов экспертных блоков
- Механизмы динамической балансировки нагрузки между репликами экспертов
- Оптимизации GPU-вычислений

Экспериментальные оценки показали, что предложенный подход обеспечивает возможность динамической балансировки нагрузки между экспертными блоками и сохраняет доступность при их частичных отказах. Полученные результаты демонстрируют практическую применимость клиент-серверного подхода для построения гибких систем инференса МоЕ моделей.

Структура работы

Работа состоит из введения, трех глав и заключения.

В первой главе приводится обзор литературы и существующих подходов к инференсу больших языковых моделей. В ней рассматриваются архитектура трансформера, особенности масштабирования плотных моделей, конвейерный, тензорный и экспертный параллелизм, архитектура Mixture-of-Experts, коллективная операция All-to-All, а также ограничения коллективных решений с точки зрения балансировки нагрузки и обработки частичных отказов.

Во второй главе описывается предложенный метод: клиент-серверная архитектура инференса МоЕ моделей, устройство программных компонентов МоЕ инференса и подсистемы передачи тензоров, схема взаимодействия основной части модели с экспертными блоками, механизм асинхронной диспетчеризации, мониторинг состояния экспертных блоков и динамическая балансировка нагрузки. Также в главе рассматриваются оптимизации, выполненные в ходе реализации: уменьшение объема передаваемых по сети данных, оптимизация передачи тензоров, выравнивание экспертных активаций и применение групповых матричных умножений. Архитектура проекта и основные компоненты метода иллюстрируются схемами.

В третьей главе приводятся эксперименты, направленные на оценку производительности предложенного подхода, стоимости передачи тензоров, эффективности оптимизаций, возможности динамической балансировки нагрузки и поведения системы при отказах экспертных блоков. В ней фиксируется компромисс между снижением абсолютной производительности и повышением гибкости управления экспертными блоками.

Глава 1. Обзор литературы

1.1. Большие языковые модели и инференс

Большая языковая модель — это нейронная модель, обученная на больших корпусах текстовых и близких к ним символьных данных и предназначенная для обработки последовательностей. Современные большие языковые модели основываются на архитектуре трансформера, предложенной в работе “Attention Is All You Need” [6]. В отличие от рекуррентных архитектур, трансформер использует механизм внимания, что позволяет эффективно учитывать зависимости между токенами и хорошо распараллеливать вычисления на GPU.

Рост качества больших языковых моделей тесно связан с масштабированием числа параметров, объема обучающих данных и вычислительного бюджета. На примере GPT-3 было показано, что увеличение масштаба авторегрессионной языковой модели приводит к появлению новых возможностей и повышению качества на широком наборе задач [7]. Работы о формализации масштабирования моделей и вычислительно оптимальном обучении показывают, что качество языковой модели зависит не только от размера модели, но и от соотношения между числом ее параметров и объемом данных [4; 5]. Это объясняет, почему развитие больших языковых моделей постоянно упирается в системные ограничения: память GPU, пропускную способность между устройствами и стоимость выполнения.

Жизненный цикл большой языковой модели делится на два этапа: обучение и вывод. На этапе обучения параметры модели обновляются на основе больших наборов данных. Этот этап требует огромного числа прямых и обратных проходов, хранения градиентов и промежуточных активаций. Поэтому для обучения особенно важны масштабирование на многие GPU, быстрый обмен градиентами и возможность долго выполнять вычисления без отказов.

Этап вывода, или инференса, имеет другую природу. Параметры модели уже зафиксированы, а система должна отвечать на пользовательские запросы с ограничениями по задержке, пропускной способности и стабильности. Для авторегрессионных языковых моделей инференс выполняется последовательно по токенам: модель получает текущий контекст, вычисляет распределение следующего токена, добавляет выбранный токен в контекст и повторяет процесс. Поэтому даже небольшая задержка одного шага умножается на число токенов в выходной последовательности. На практике важны не только средняя скорость, но и верхние перцентили задержки, так как пользовательский запрос завершается только после выполнения всей цепочки шагов.

Авторегрессионная природа инференса делает один шаг коротким, но многократно повторяемым. На каждом шаге запускаются операции на GPU, выполняются обращения к KV кэшу, а в распределенной системе еще и передаются промежуточные тензоры между устройствами. Поэтому существенной частью задержки становится не только арифметика внутри слоев, но и накладные расходы запуска операций и согла-

сования работы нескольких GPU.

Инференс крупных моделей осложняется тем, что веса модели, KV кэш при длинных контекстах и промежуточные активации при большом числе параллельных запросов могут не помещаться в память одного GPU. Даже если модель помещается на одно устройство, один GPU может не обеспечить требуемую пропускную способность при большом числе параллельных запросов. Поэтому промышленные системы инференса используют несколько устройств и совмещают вычислительные оптимизации, управление памятью и распределенное выполнение. Например, DeepSpeed Inference рассматривает инференс трансформеров как системную задачу, в которой нужно одновременно учитывать размер модели, задержку, пропускную способность, оптимизированные ядра и размещение данных между GPU, CPU и накопителями [1].

1.2. Архитектура Mixture-of-Experts

Один из способов масштабировать языковые модели — увеличивать плотную модель, в которой каждый вход проходит через все имеющиеся параметры. Такой подход прост с точки зрения архитектуры, но дорог при инференсе: каждый прямой проход использует весь набор весов. Поэтому увеличение плотной модели почти напрямую повышает стоимость одного шага инференса.

Архитектура Mixture-of-Experts, или MoE, предлагает другой путь. В ней полностью линейные слои (feed-forward network) в трансформере заменяются наборами экспертов, а отдельный роутер выбирает, какие эксперты должны обработать конкретный токен или группу токенов. В работе Shazeer et al. был предложен sparsely-gated MoE слой, где для каждого входа активируется только небольшое число экспертов [3]. За счет этого можно увеличить общее число параметров модели, но не использовать все экспертные параметры при каждом проходе.

Главное преимущество архитектуры MoE состоит в разделении двух величин: общей емкости модели и числа операций на один вход. Модель может содержать много экспертов и, следовательно, иметь большую параметрическую емкость, но конкретный токен обрабатывается только выбранным подмножеством экспертов. Это делает MoE привлекательным направлением развития больших языковых моделей: оно позволяет увеличивать масштаб модели без такого же резкого роста вычислительной стоимости инференса.

Эта идея получила развитие в крупных трансформерных моделях. GShard использует условное вычисление и автоматическое разбиение вычислений для масштабирования MoE трансформеров до сотен миллиардов параметров [17]. Switch Transformer упрощает маршрутизацию, выбирая одного эксперта для каждого токена, и показывает возможность масштабирования до триллионного числа параметров при контролируемой вычислительной стоимости [9]. GLaM также использует разреженно активируемую архитектуру MoE и показывает, что она может увеличивать емкость языковой модели при меньшей стоимости обучения по сравнению с плотными вариантами сопоставимого

качества [10]. DeepSpeed-MoE рассматривает уже не только обучение, но и инференс MoE моделей, подчеркивая, что для практического применения нужны специальные системные решения [14].

При этом MoE не устраняет системные сложности, а переносит их в другую часть системы. Вместо одной плотной операции возникает маршрутизация токенов, распределение нагрузки между экспертами, отправка активаций к выбранным экспертам и сбор результатов. Если эксперты размещены на разных GPU или узлах, MoE слой становится не только вычислительной, но и коммуникационной задачей.

1.3. Методы распределенного инференса

Распределенный инференс нужен в двух основных случаях. Первый случай — модель или ее рабочее состояние не помещаются в память одного GPU. Второй случай — одного устройства недостаточно по скорости, даже если памяти хватает. В обоих случаях модель, входные данные или поток запросов распределяются между несколькими устройствами, а система должна согласовать вычисления и обмен промежуточными результатами.

Параллелизм по данным применяется, когда нужно обслуживать больше независимых запросов без изменения устройства одной копии модели. В этом случае несколько копий модели запускаются на разных GPU, а пользовательские запросы распределяются между этими копиями. Такой подход хорошо работает, когда модель помещается в память одного устройства, однако не помогает, если модель слишком большая для одной видеокарты.

Тензорный параллелизм применяется, когда отдельные матричные операции внутри слоя разбиваются между несколькими GPU. Например, части весов могут храниться на разных устройствах, каждое устройство считает свою часть результата, после чего результаты объединяются с помощью коллективной операции (All-Reduce). Этот подход является стандартным способом распределенного инференса крупных моделей. В статье Megatron-LM такая схема описана для разбиения на уровне слоя модели на множестве GPU [11]; та же идея используется и в системах инференса, где нужно уменьшить объем памяти и вычислений на одном устройстве.

Конвейерный параллелизм размещает группы слоев на разных устройствах. Когда одна группа слоев обрабатывает вход, она передает активации следующей группе, и так далее. Такой подход снижает требования к памяти одного GPU, но добавляет зависимость между стадиями. Для обучения конвейер можно заполнять микробатчами, а при инференсе отдельных запросов он потенциально может увеличивать задержку, так как запрос должен пройти последовательную цепочку групп слоев.

Экспертный параллелизм характерен именно для MoE моделей. В нем разные эксперты размещаются на разных GPU или узлах, а токены после маршрутизации отправляются к тем экспертам, которые были выбраны роутером. В отличие от тензорного параллелизма, где коммуникационная схема часто регулярна и связана со структурой

слоя, в экспертном параллелизме трафик зависит от входных данных. Это делает нагрузку менее предсказуемой: один эксперт может получить много токенов, другой — мало или не получить ни одного.

На практике эти методы часто комбинируются. Например, плотные части модели могут использовать тензорный или конвейерный параллелизм, а экспертные слои — экспертный параллелизм. Такой гибридный подход нужен, потому что разные части модели имеют разные ограничения: где-то не хватает памяти, где-то требуется ускорить матричные операции, а где-то главной проблемой становится обмен активациями между экспертами.

1.4. Коллективные коммуникации и ограничения распределенного инференса

Распределенный инференс невозможен без коммуникационных примитивов, то есть операций обмена данными между процессами-участниками. В литературе и практических системах широко используются коллективные операции, например Broadcast, Gather, AllReduce, AllGather, ReduceScatter и All-to-All. На NVIDIA GPU такие операции обычно реализуются с опорой на библиотеки из экосистемы NVIDIA. NCCL предоставляет коллективные и point-to-point операции для GPU [15]. NVSHMEM реализует модель OpenSHMEM для кластеров NVIDIA GPU, поддерживает односторонний доступ к памяти, синхронизацию и коллективные операции из CPU, CUDA streams и непосредственно из CUDA kernels [16]. Эти библиотеки важны не как самостоятельные архитектуры инференса, а как практическая основа, на которой строятся как обобщенные коллективные операции, так и более специализированные MoE коммуникации, включая dispatch/combine в библиотеках экспертного параллелизма. Для тензорного параллелизма типичны AllReduce, AllGather и ReduceScatter, а для экспертного параллелизма особенно важен All-to-All: каждый участник может отправить часть токенов каждому другому участнику, владеющему нужными экспертами.

Коллективные операции удобны тем, что дают высокопроизводительную абстракцию над сложной топологией GPU и сети. Разработчику не нужно вручную описывать каждую отдельную передачу: достаточно задать операцию для группы участников, а библиотека выбирает алгоритмы и транспорт. Для синхронного выполнения на фиксированном наборе устройств это естественная модель: все участники входят в одну коммуникационную группу, вызывают одну и ту же коллективную операцию и продолжают работу после ее завершения.

Однако именно эта связанность становится ограничением для задач, где решение о выборе исполнителя должно приниматься во время работы. Коллективная операция предполагает согласованное участие процессов из заданной группы. Если один участник задержался или стал недоступен, вся операция может задержаться или завершиться ошибкой. Современные библиотеки добавляют средства обработки ошибок, завершения коммуникационных групп и создания новых групп, но это не превращает

коллективную операцию в механизм динамической балансировки нагрузки. Изменение состава участников требует явного управления группами и согласования состояния между оставшимися процессами [15].

Для инференса МоЕ моделей это особенно важно. В классической схеме экспертного параллелизма все GPU с экспертами участвуют в операции All-to-All. Такая схема эффективна, когда все участники исправны, а нагрузка на отдельные модули постоянна и распределена равномерно. Однако маршрутизация токенов в МоЕ зависит от входных данных, поэтому разные эксперты могут получать существенно разное число токенов. Показательно, что для этой проблемы существуют отдельные системные решения, например EPLB (Expert Parallelism Load Balancer). EPLB балансирует экспертный параллелизм за счет выбора числа реплик экспертов и их размещения по GPU и узлам на основе измеренной нагрузки [18].

Однако такая балансировка остается привязанной к заранее выбранной раскладке экспертов по рангам. Распределение входных запросов может меняться во времени, очереди у исполнителей могут расти неравномерно, а часть экспертных блоков может временно становиться недоступной или перегруженной. В этих условиях статически выбранное размещение экспертов остается лишь приближением к текущей нагрузке. При такой неравномерности быстрые участники вынуждены ждать медленных, а если экспертный блок нужно временно исключить или добавить во время работы новую реплику, то фиксированная коллективная схема не удовлетворяет таким требованиям. Если же один и тот же эксперт размещен в нескольких экспертных блоках, клиент-серверная схема позволяет трактовать эти блоки как адресуемые реплики и выбирать менее загруженную доступную реплику во время обработки запроса.

1.5. ДеерЕР и коллективы для МоЕ

Для МоЕ существуют и более специализированные реализации коллективного подхода. Обычная All-to-All операция передает наборы тензорных фрагментов между всеми участниками, но МоЕ слой имеет более конкретную структуру: сначала выполняется `dispatch`, то есть отправка токенов к выбранным экспертам, затем вычисление экспертов, затем `combine`, то есть сбор результатов обратно в исходный порядок. Поэтому специализированная библиотека может оптимизировать не абстрактный All-to-All примитив, а именно пару `dispatch/combine` с учетом формы тензоров, числа экспертов, `top-k` маршрутизации, точности данных и используемой межузловой или внутрисерверной связи.

Одной из таких реализаций является ДеерЕР. Библиотека ДеерЕР сфокусирована на экспертном параллелизме и предоставляет высокопроизводительные GPU-ядра для МоЕ `dispatch` и `combine`. Для нее важны режимы высокой пропускной способности и низкой задержки, поддержка низкой точности, включая FP8, а также применимость как к обучению, так и к инференсу современных МоЕ моделей [2]. ДеерЕР использует NVSHMEM как одну из низкоуровневых основ для коммуникаций между участника-

ми и остается примером специализированной реализации МоЕ коммуникаций поверх низкоуровневого слоя обмена данными.

В данной работе ДеерЕР важен как представитель коллективного подхода к МоЕ инференсу. Предлагаемый метод противопоставляет две архитектурно разные схемы: синхронную схему экспертного параллелизма на коллективах и клиент-серверную схему с независимыми экспертными блоками. Поэтому для экспериментального сравнения не требуется сравниваться со всеми возможными реализациями All-to-All. Достаточно выбрать сильную специализированную реализацию, которая уже оптимизирована именно под МоЕ dispatch/combine и может рассматриваться как практически готовый к промышленному использованию ориентир для коллективного подхода. В этой роли ДеерЕР подходит лучше, чем простой All-to-All через обобщенную коммуникационную библиотеку, поскольку он ближе к тому, как МоЕ коммуникации оптимизируются в реальных системах.

1.6. Движки адресной передачи памяти: NIXL и MoonCake Transfer Engine

Переход от коллективной схемы МоЕ к клиент-серверной архитектуре не устраняет необходимость передавать данные между GPU. Наоборот, эта необходимость становится явной частью проектируемой системы. В коллективном подходе операция All-to-All не только задает синхронизацию участников, но и берет на себя перемещение байтов: активации доставляются к GPU с нужными экспертами, а результаты возвращаются обратно. Если заменить такую операцию на обращение основной части модели к конкретному экспертному блоку, то система должна самостоятельно решить, как эффективно передать область GPU-памяти от одного исполнителя к другому и затем получить результат.

Обычного управляющего протокола для этого недостаточно. Он может передать команду, идентификатор запроса или метаданные, но сами активации и результаты экспертных вычислений являются буферами в памяти GPU. Их невыгодно переносить через лишние промежуточные копирования и синхронные операции на CPU: при каждом обращении к экспертному блоку это напрямую увеличивает задержку МоЕ слоя. Поэтому в предлагаемой архитектуре нужен отдельный слой адресной передачи памяти между GPU.

Для обмена данными между видеокартами существует несколько уровней программных средств. На нижнем уровне находятся физические каналы и сетевые технологии: PCIe, NVLink [19] и InfiniBand. Выше располагаются транспортные библиотеки, например UCX, которые скрывают детали конкретного канала и позволяют работать с памятью CPU и GPU через общий интерфейс [20]. Еще выше находятся специализированные движки передачи данных. В контексте данной работы они нужны не как самостоятельная система инференса, а как замена той части коллективной операции, которая обеспечивала собственно перемещение буферов между GPU.

NIXL (NVIDIA Inference Xfer Library) является одной из реализаций такого слоя. Он используется в экосистеме NVIDIA Dynamo и ориентирован на перемещение данных в распределенном инференсе моделей [21]. Dynamo — открытый фреймворк для распределенного инференса LLM; NVIDIA представляет его как промышленную основу для масштабируемого инференса и указывает интеграции с TensorRT-LLM, SGLang и vLLM [22]. В Dynamo NIXL применяется как механизм передачи KV кэша между исполнителями в disaggregated serving [23]. Для предлагаемого МоЕ инференса важен тот же принцип: участники заранее обмениваются метаданными о доступных областях памяти, регистрируют буферы и затем выполняют адресные операции чтения или записи между конкретными сторонами. Такая модель позволяет основной части модели отправлять активации выбранному экспертному блоку, не вовлекая в обмен всю коллективную группу.

MoonCake Transfer Engine — другая реализация того же класса задач. Она появилась в контексте систем обслуживания больших языковых моделей и используется как слой передачи данных в проекте MoonCake. Для нее важна поддержка разных протоколов: TCP, RDMA, NVMe-oF, а также транспортов на основе NVLink для связи внутри сервера и между связанными узлами [24]. В отличие от обычного обмена сообщениями, Transfer Engine предоставляет операции чтения и записи фрагментов DRAM/VRAM через выбранный транспорт [25]. Поэтому его можно использовать как основу для передачи памяти между основной частью модели и экспертными блоками, а не только управляющих сообщений.

В данной работе NIXL и MoonCake Transfer Engine рассматриваются как две конкретные реализации слоя адресной передачи памяти между GPU. Их использование позволяет отделить клиент-серверную архитектуру МоЕ слоя от конкретного механизма перемещения данных. Если метод работает только с одним движком, трудно понять, является ли результат следствием архитектуры или частной реализации передачи.

Выводы по главе

В главе показано, что для инференса больших языковых моделей особенно важны задержка, пропускная способность, работа с памятью GPU и способность обслуживать поток запросов без падения всей системы. По мере роста моделей распределенный инференс становится не оптимизацией, а необходимым условием практического применения.

Архитектура МоЕ является одним из наиболее перспективных направлений развития больших языковых моделей. Но эта выгода достигается ценой усложнения маршрутизации и коммуникаций.

Традиционные коллективные коммуникации хорошо подходят для синхронного выполнения на фиксированной группе участников, но плохо сочетаются с требованием динамически выбирать менее загруженную реплику эксперта, исключать недоступные экспертные блоки и добавлять новых исполнителей во время работы. Поэтому

для предлагаемого метода важен слой адресной передачи GPU-памяти: после отказа от All-to-All именно он должен обеспечить эффективное перемещение активаций и результатов между основной частью модели и выбранными экспертными блоками. NIXL и MoonCake Transfer Engine дают такую основу и позволяют строить архитектуру, в которой экспертные блоки являются более независимыми, наблюдаемыми и управляемыми компонентами распределенного инференса.

Глава 2. Метод

2.1. Описание метода

Предлагаемый метод инференса МоЕ моделей основан на переходе от жесткой коллективной схемы взаимодействия к клиент-серверной архитектуре. В коллективной схеме обмен активациями между рангами задается общей операцией All-to-All: все участники входят в заранее фиксированную группу и синхронно выполняют операции `dispatch` и `combine`. Такой подход хорошо согласуется с моделью статической раскладки экспертов по видеокартам, однако хуже подходит для сценариев, в которых нагрузка на экспертов меняется во время инференса, а клиентские экспертные блоки могут добавляться, исключаться или временно становиться недоступными.

В предлагаемой архитектуре МоЕ слой разделяется на сервер-оркестратор и набор удаленных экспертных блоков. Сервером является компонент `DistMoEBlock`: он выполняет роутинг, формирует задания `dispatch`, выбирает физический экспертный блок для обработки, управляет своими буферами обмена и выполняет операцию `combine` после получения результата. Клиентскую часть реализует компонент `DistExpertsBlock`. Он хранит локальное подмножество экспертов и выполняет вычислительные задания, поступающие от серверов-оркестраторов.

Различие между коллективной схемой и предлагаемой архитектурой удобно рассмотреть на одном цикле `dispatch/combine`. В коллективном подходе (рис. 1) оба этапа выполняются всей группой участников. В предлагаемом методе (рис. 2) `dispatch` выражается как адресное задание выбранному экспертному блоку, а `combine` выполняется тем же сервером-оркестратором после получения результата от клиента.

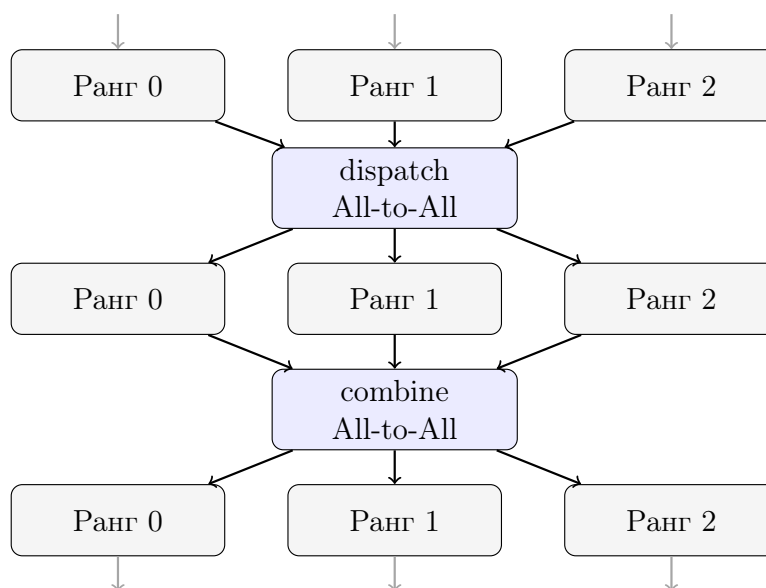


Рис. 1 — Один цикл `dispatch/combine` в коллективной схеме

Ключевое следствие такого разделения состоит в том, что выбор исполнителя становится частью логики инференса, а не только следствием заранее заданной кол-

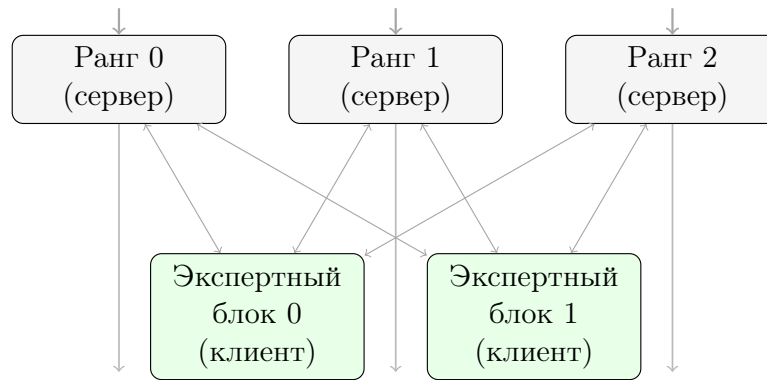


Рис. 2 — Один цикл dispatch/combine в предлагаемой архитектуре распределенного МоЕ слоя

лективной конфигурации. Если эксперт представлен несколькими физическими репликами, сервер-оркестратор может выбрать одну из них с учетом текущей доступности и очереди заданий. При этом клиентские части могут создаваться, удаляться или временно исключаться прямо во время работы системы: сервер обновляет пул доступных экспертных блоков и планирует последующие задания уже с учетом нового состава исполнителей.

Для передачи активаций и результатов вычислений между сервером и клиентами используется абстракция **TensorTransferEngine** (ТТЕ). Она связывает управляющую часть МоЕ слоя с конкретным механизмом адресной передачи GPU-памяти. Управляющие сообщения передают метаданные задания, а сами активации и результаты размещаются в заранее зарегистрированных буферах, доступных для чтения и записи между удаленными устройствами.

Во время одного вызова МоЕ слоя сервер сначала вычисляет значения роутера и определяет $\text{top-}k$ экспертов для входных активаций. Затем активации группируются по экспертам, для каждой порции создается задание обработки, выбирается экспертный блок и выполняется dispatch. Удаленный экспертный блок читает входные данные из серверного буфера, выполняет локальные экспертные вычисления, записывает результат в выходной буфер сервера и отправляет уведомление о завершении. После этого сервер выполняет combine: считывает готовую часть результата и добавляет ее в выходной батч слоя с учетом весов роутинга.

2.2. Архитектура метода

На рисунке 3 показана общая структура предлагаемой архитектуры. Диаграмма отделяет серверную часть от клиентской и показывает только основные компоненты: оркестратор МоЕ слоя, пул экспертных блоков, задачи обработки, пулы буферов и экземпляры ТТЕ на обеих сторонах взаимодействия.

В этой структуре управляющий контур (control plane) и контур передачи данных (data plane) разделены. Управляющий контур отвечает за выбор исполнителя, сериализацию метаданных и уведомления о состоянии задания. Контур передачи данных

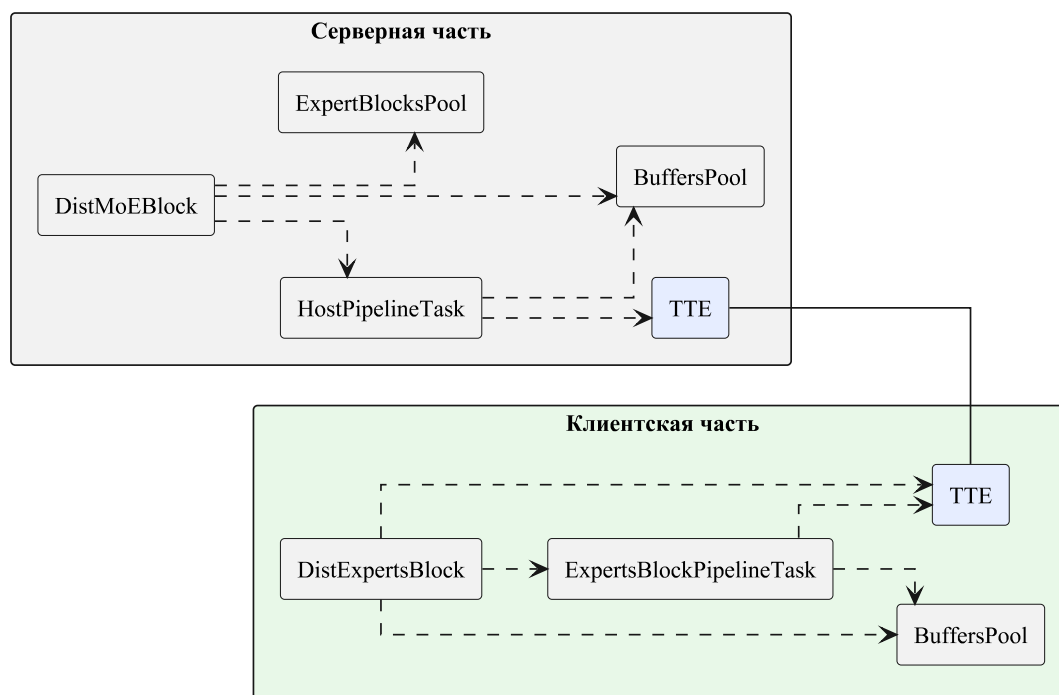


Рис. 3 — UML-диаграмма основных компонентов

работает с зарегистрированными областями памяти: экспертные активации читаются экспертным блоком из зарезервированного серверного буфера, а результат вычислений записывается обратно в выходной буфер, зарезервированный на сервере для данного dispatch.

Диаграмма последовательности одного цикла dispatch/combine приведена на рисунке 4. Она фиксирует порядок операций на уровне программных компонентов: подготовку метаданных по экспертам, создание конвейерной задачи, выбор экспертного блока, отправку управляющего сообщения `DispatchSubmit`, создание клиентской задачи, чтение данных с сервера, экспертные вычисления, запись результата и обратное уведомление о готовности выходного буфера, после которого запускается операция combine.

Пулы буферов используются на обеих сторонах взаимодействия. На сервере они ограничивают число одновременно выполняемых передач и связывают каждое задание с конкретным входным и выходным буфером. На клиенте они позволяют переиспользовать локальные области памяти для чтения входных данных и записи результата. Такое устройство передаваемой памяти дает возможность организовать конвейерное выполнение: пока один буфер используется для экспертных вычислений, другой может участвовать в чтении следующей порции активаций или записи уже готового результата. За счет этого операции чтения и записи могут перекрываться с вычислениями, а канал передачи данных и шина памяти используются более эффективно, что будет показано в экспериментах.

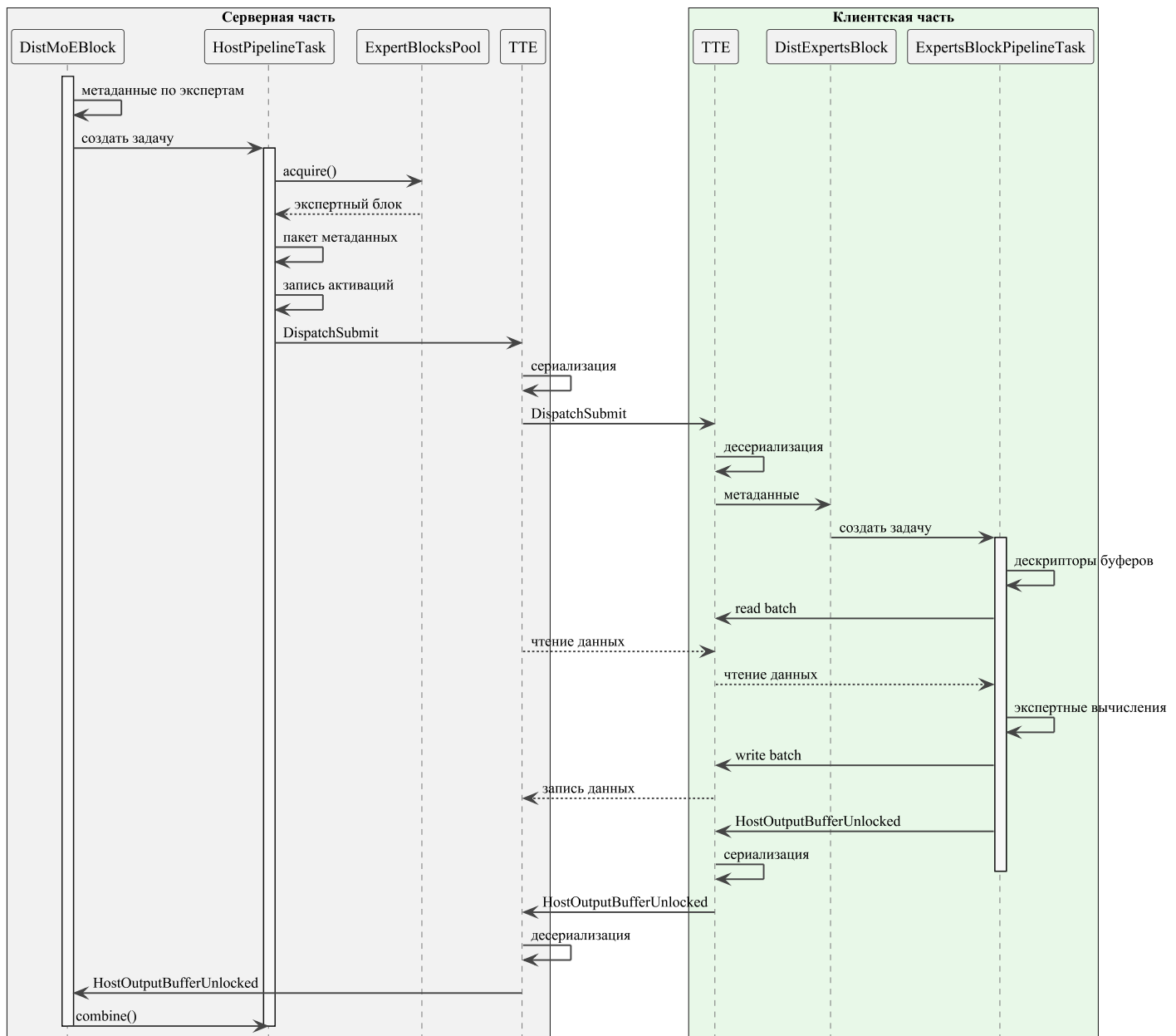


Рис. 4 — UML-диаграмма последовательности одного цикла dispatch/combine

2.3. Компонент DistMoEBlock

Компонент `DistMoEBlock` реализует серверную часть распределенного MoE слоя. Его ответственность состоит в оркестрации всего распределенного MoE слоя: подготовить входной батч к обработке экспертами, распределить работу по удаленным блокам, принять готовые результаты и агрегировать их в выходной батч слоя.

На рисунке 5 показаны основные связи серверной части. `DistMoEBlock` содержит роутер `gate`, метаданные экспертов, пул доступных экспертных блоков, два пула буферов и серверный экземпляр TTE. Сначала роутер вычисляет веса и индексы $\text{top-}k$ экспертов. Для каждого выбранного эксперта компонент хранит индексы токенов, веса роутинга и очередь еще не обработанных отрезков активаций.

Dispatch на серверной стороне выполняется через `HostPipelineTask`. Задача вы-

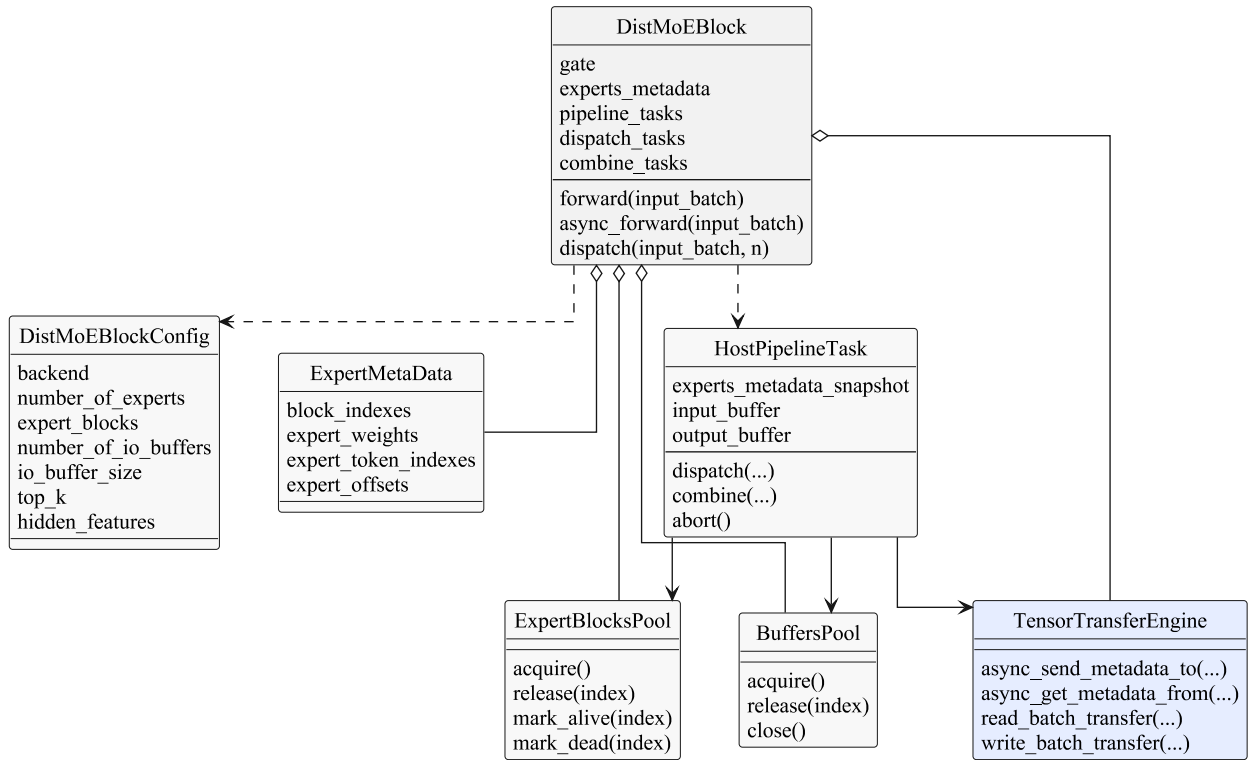


Рис. 5 — UML-диаграмма классов серверной части **DistMoEBlock**

бирает экспертный блок из **ExpertBlocksPool**, формирует пакет метаданных по экспертам, резервирует входной и выходной буферы на стороне сервера, копирует нужные активации в зарезервированный входной буфер и отправляет управляющее сообщение **DispatchSubmit**. При подтверждении готовности результата сервер вызывает **combine** для соответствующей задачи: данные из выходного буфера добавляются в общий выходной батч с учетом весов роутинга.

Отдельная часть ответственности **DistMoEBlock** связана с динамическим состоянием экспертных блоков. Компонент поддерживает фоновые проверки доступности, умеет помечать блоки как доступные или недоступные и отменять незавершенные задания, отправленные на блок, ставший недоступным. При отмене задания его отрезки активаций возвращаются в очереди обработки, поэтому дальнейший **dispatch** может быть направлен на другой доступный экспертный блок с нужными экспертами. Эти служебные механизмы на диаграмме не детализированы, чтобы сохранить фокус на основных компонентах серверной части.

2.4. Компонент **DistExpertsBlock**

Компонент **DistExpertsBlock** реализует клиентскую часть архитектуры. Его задача состоит в том, чтобы принять задание от сервера, выполнить локальные экспертные вычисления, записать полученный результат в серверный буфер и сообщить серверу о завершении задачи.

На рисунке 6 приведена структура клиентской части. Компонент хранит локальные экспертные слои, пулы локальных входных и выходных буферов, а также дескрип-

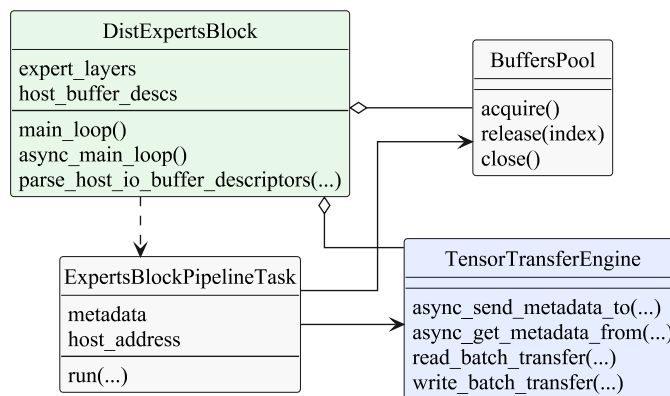


Рис. 6 — UML-диаграмма классов клиентской части `DistExpertsBlock`

торы серверных буферов, полученные во время инициализации. Основной цикл компонента принимает управляющие сообщения от сервера: сообщение с дескрипторами серверных буферов обновляет локальные таблицы, а сообщение `DispatchSubmit` приводит к созданию клиентской задачи обработки.

`ExpertsBlockPipelineTask` разбирает метаданные задания, получает дескрипторы серверных буферов и резервирует локальные буферы. Затем задача выполняет чтение входного батча через ТТЕ, запускает локальные экспертные вычисления и записывает результат обратно в серверный выходной буфер. После успешной записи клиент отправляет серверу уведомление `HostOutputBufferUnlocked`; это уведомление означает, что серверный выходной буфер содержит готовую часть результата и может быть использован на этапе `combine`.

Вычислительная часть поддерживает два режима. Если доступен `grouped GeMM` и он разрешен для задания, вычисления по нескольким экспертам выполняются через групповую матричную операцию. В противном случае используется последовательный проход по экспертам внутри блока. Оба режима сохраняют одну и ту же внешнюю семантику: клиент получает входные активации, применяет соответствующие экспертные слои и записывает выход в буфер, указанный сервером.

2.5. Компонент `TensorTransferEngine`

`TensorTransferEngine` является связующей компонентой распределенного МоЕ слоя. В архитектуре существует отдельный экземпляр ТТЕ на стороне сервера и отдельные экземпляры на стороне клиентских блоков; они обмениваются управляющими сообщениями и выполняют адресные операции чтения и записи между зарегистрированными областями памяти.

На рисунке 7 показано место ТТЕ в реализации передачи данных. Базовый класс задает общий интерфейс для регистрации и снятия регистрации памяти, сериализации и десериализации дескрипторов, отправки и приема управляющих сообщений, запуска операций чтения и записи, а также ожидания завершения асинхронных передач. Реализации на основе NIXL и MCTE используют этот интерфейс для подключения движков

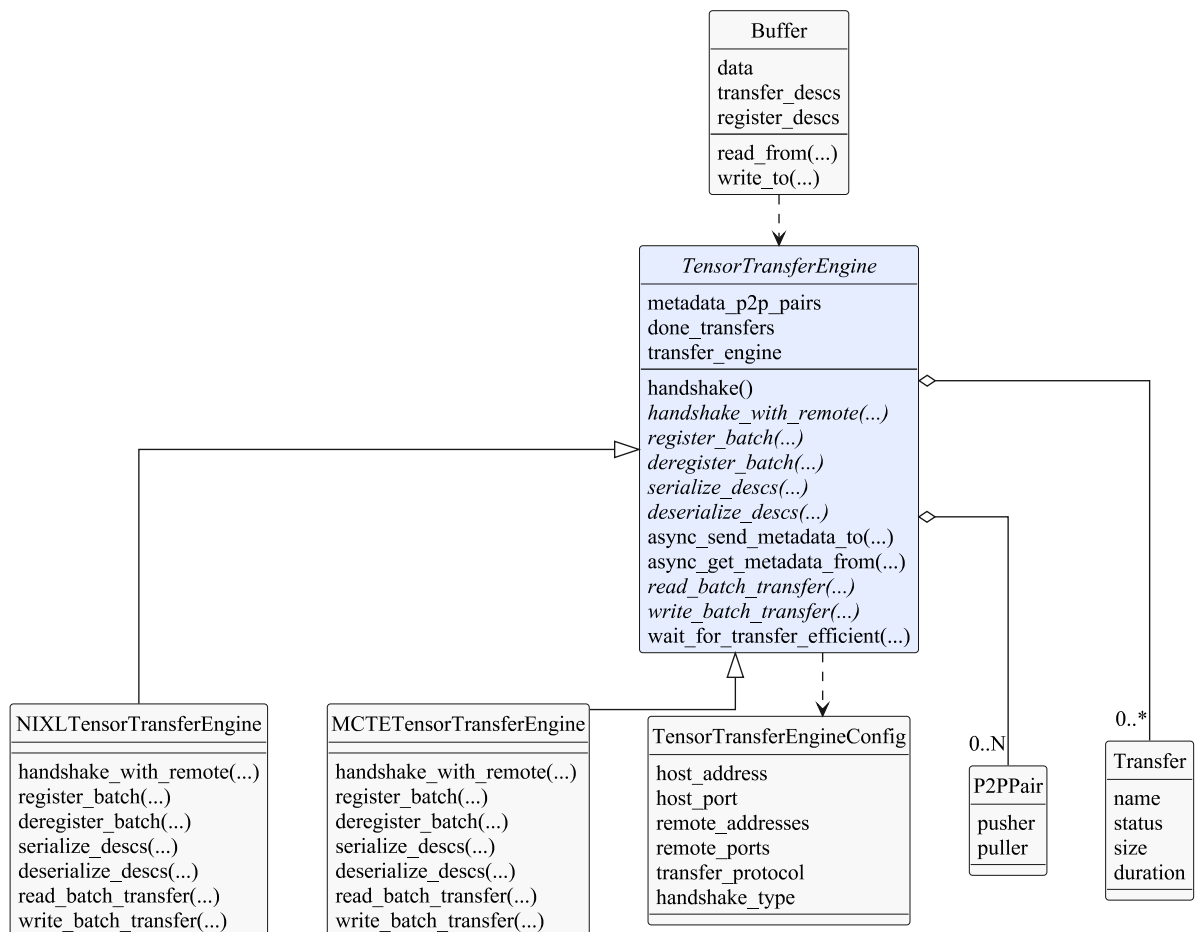


Рис. 7 — UML-диаграмма классов компонента **TensorTransferEngine**

адресной передачи данных.

Буфер в этой архитектуре является зарегистрированной областью памяти на конкретном устройстве. При создании буфера ТТЕ регистрирует его данные, получает дескрипторы передачи и сериализует их для передачи на экспертные блоки. Поэтому серверная и клиентская задачи работают не с низкоуровневыми деталями конкретного движка, а абстракциями более высокого уровня: буферами и их дескрипторами.

Фоновый мониторинг передач внутри ТТЕ отслеживает состояние запущенных операций и позволяет задачам ожидать завершения конкретного чтения или записи. Это важно для корректного конвейера операций `dispatch/combine`: экспертные вычисления должны начинаться только после завершения чтения входного батча, а уведомление `HostOutputBufferUnlocked` должно отправляться только после завершения записи результата в серверный буфер.

Выводы по главе

В главе представлен метод, в котором МоЕ слой разделяется на оркестратор и набор удаленных экспертных блоков. Оркестратор (сервер) принимает решения о роутинге, выборе исполнителей, постановке заданий и объединении результатов, а экспертные блоки (клиенты) выполняют локальные вычисления над переданными активациями.

Ключевое отличие предложенного подхода от коллективной схемы состоит в том, что обработка активаций выражается через адресные задания конкретным вычислительным модулям. Это создает основу для динамической балансировки нагрузки, повторного выбора физической реплики эксперта и продолжения инференса при недоступности части экспертных блоков.

Глава 3. Эксперименты

В данной главе описывается экспериментальная оценка прототипа распределенного МоЕ слоя на клиент-серверной архитектуре. Цель экспериментов состоит в том, чтобы сравнить производительность реализованного подхода с решениями, основанными на коллективных коммуникациях, а также проверить динамическую балансировку нагрузки и устойчивость системы к отказам экспертных блоков.

3.1. Методика экспериментальной оценки

Таблица 1 — Конфигурация экспериментального стенда

Параметр	Значение
GPU	8× NVIDIA H100
Объем видеопамяти	80 GB
Связь между видеокартами	InfiniBand

3.2. Производительность МоЕ инференса

В данном эксперименте измерялась производительность одного распределенного МоЕ слоя при передаче данных по RDMA с одним экспертным блоком [26]. Конфигурация слоя взята из архитектуры моделей семейства Qwen3 и включала 128 экспертов, $\text{top-}k = 8$, размер скрытого состояния 2048, промежуточный размер экспертного блока 6144 и длину последовательности 200 [27]. Для каждого размера батча выполнялось 16 прогревочных запусков и 32 запуска с измерением. Размер батча изменялся от 1000 до 9000 токенов с шагом 200 токенов.

Сравнивались четыре конфигурации реализации: базовая версия без дополнительных оптимизаций, версия с выравниванием экспертных активаций, версия с grouped GeMM и версия, в которой одновременно включены выравнивание и grouped GeMM. Измерения проводились отдельно для двух движков передачи данных: NIXL и MoonCake Transfer Engine [21; 25]. Также на графиках задержки приведен ориентир на коллективную реализацию DeepEP [2].

На графиках представлены два типа измерений. Первый соответствует случайной инициализации весов роутера, при которой входной поток распределяется между практически всеми экспертами. Второй соответствует весам роутера из модели Qwen3-30B-A3B; в этом случае активируется небольшая часть экспертов, порядка 10% от их общего числа [27].

Для графиков задержки дополнительно показан разброс измерений: нижняя и верхняя границы области соответствуют 1-му и 99-му перцентилям.

Рисунок 8 показывает, что grouped GeMM наиболее заметно снижает задержку в сценарии с большим числом активных экспертов. В этом режиме на каждого эксперта

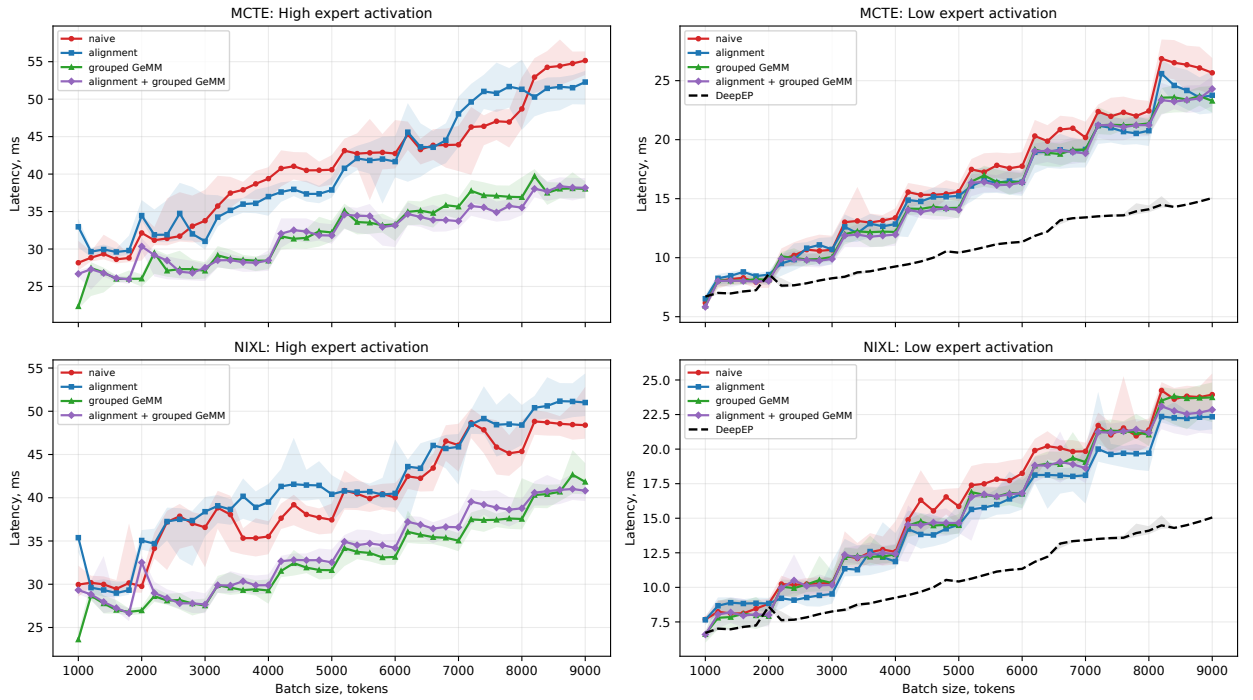


Рис. 8 — Сравнение задержки MoE слоя при использовании RDMA

приходится меньше активаций, поэтому буфер, обрабатываемый экспертным блоком, разбивается на большое число небольших фрагментов. Последовательное выполнение отдельных матричных умножений в такой ситуации увеличивает накладные расходы, тогда как grouped GeMM позволяет объединить несколько экспертных вычислений в одну более крупную GPU-операцию. В сценарии с малым числом активных экспертов абсолютная задержка ниже, а различия между конфигурациями менее выражены, поскольку каждый эксперт обрабатывает более крупные фрагменты буфера.

На рисунке 9 приведена средняя пропускная способность операций переноса памяти для сценария с малым числом активных экспертов. По мере увеличения размера батча пропускная способность отдельной операции чтения или записи снижается. Это связано с тем, что в асинхронном рантайме одновременно выполняется больше конкурентных операций переноса памяти, а общая пропускная способность шины ограничена. Поэтому средняя скорость отдельного переноса уменьшается, но при этом растет совокупная загрузка канала передачи данных.

На рисунке 10 приведена средняя доля накладных расходов в общей задержке слоя. Эта величина вычисляется как разность между временем выполнения всей задачи на экспертном блоке и суммарным временем экспертных вычислений, чтения и записи данных. Результаты показывают, что более сильное дробление буфера приводит к росту доли накладных расходов. При этом с увеличением размера батча эта доля растет умеренно, что указывает на сохранение устойчивого поведения системы при росте нагрузки.

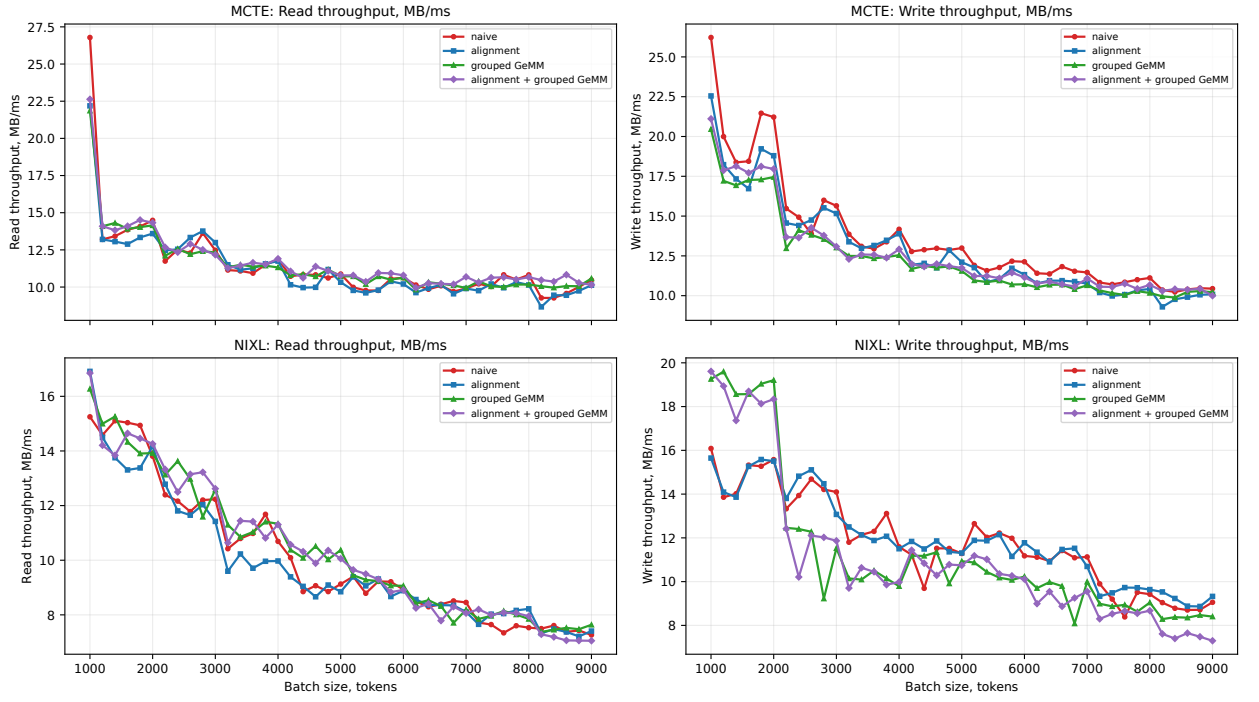


Рис. 9 — Пропускная способность чтения и записи при передаче данных МоЕ слоя по RDMA

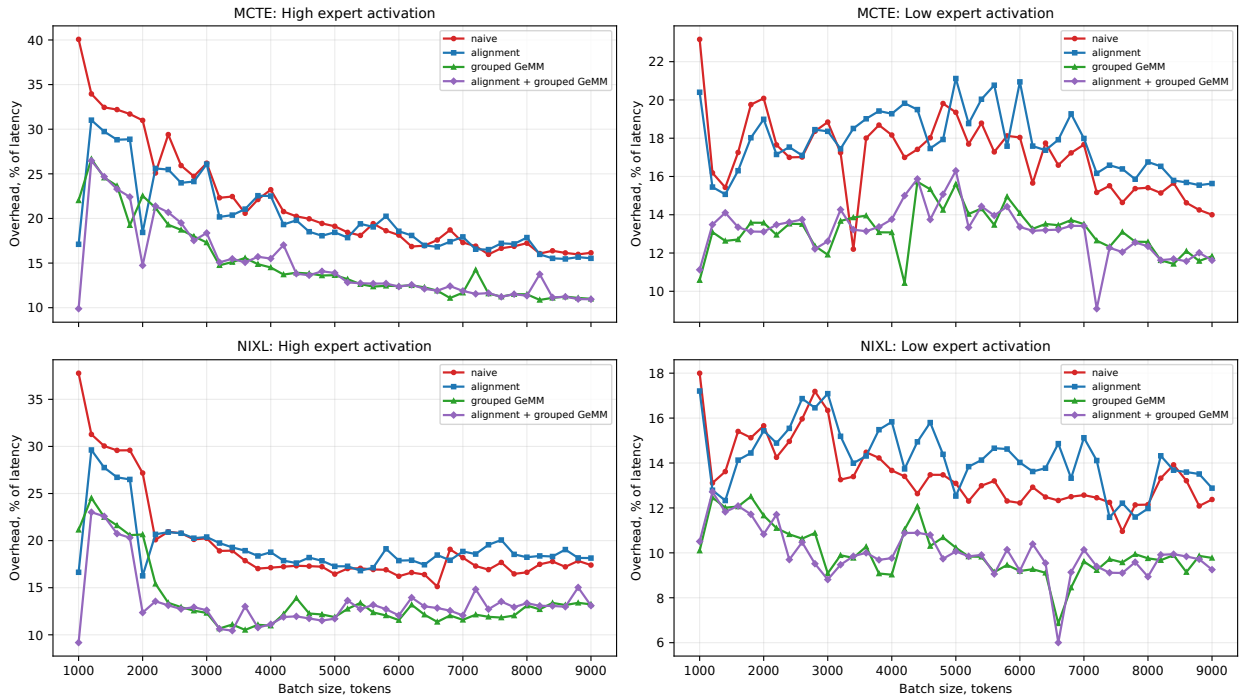


Рис. 10 — Доля накладных расходов в задержке МоЕ слоя при использовании RDMA

3.3. Балансировка нагрузки и отказоустойчивость

Для проверки устойчивости к частичным отказам был проведен эксперимент с фиксированной целевой нагрузкой 50 запросов в секунду. В предлагаемой архитектуре использовались два экспертных блока, размещенные на разных GPU; в коллективной реализации использовались два ранга. В момент времени t_1 , отмеченный красной вер-

тикальной линией, один экспертный блок исключался из пула доступных исполнителей. После этого система должна была продолжить обработку запросов на оставшемся экспертном блоке. В момент времени t_2 , отмеченный зеленой вертикальной линией, в пул добавлялся новый экспертный блок с той же конфигурацией гиперпараметров, после чего ожидалось восстановление пропускной способности до уровня, близкого к исходному. Для коллективной реализации аналогичный сценарий моделировался отказом одного ранга: после такого отказа следующий вызов `dispatch` или `combine` на оставшемся ранге приводит к ошибке, поскольку коллективная операция больше не может быть выполнена всей заданной группой участников.

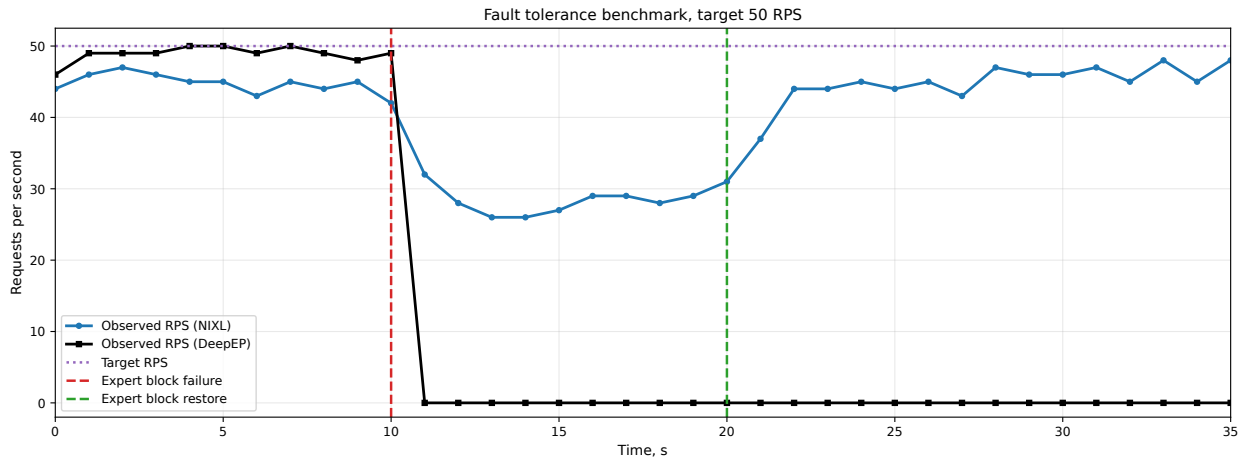


Рис. 11 — Изменение пропускной способности при временном отказе экспертного блока

На рисунке 11 показана динамика пропускной способности во времени для предлагаемой реализации и коллективной реализации DeepEP [2]. После отказа одного из двух экспертных блоков наблюдается снижение пропускной способности, однако обработка запросов не прекращается. Такое поведение соответствует цели предлагаемой архитектуры: продолжать инференс на оставшихся доступных экспертных блоках, даже при отказе одного или нескольких из них.

Для DeepEP в данном эксперименте отказ одного ранга приводит к падению пропускной способности до нуля: коллективная операция требует участия всей группы участников, поэтому отказ даже одного из них приводит к падению всей системы.

После добавления нового экспертного блока пропускная способность предлагаемой реализации возвращается к уровню, близкому к целевой нагрузке. Таким образом, эксперимент показывает практическую пользу динамического управления пулом исполнителей: система деградирует контролируемо, а не останавливает инференс целиком.

Выводы по главе

В главе оцениваются основные свойства предлагаемой архитектуры инференса МоЕ слоя: производительность, накладные расходы на клиент-серверную природу инференса, поведение при изменении нагрузки, возможность динамической балансировки нагрузки и отказоустойчивость.

Ожидаемый итог экспериментальной части состоит в следующем: предложенный подход уступает высоко оптимизированным коллективным решениям по пиковой производительности, но обеспечивает динамическую балансировку нагрузки и сохранение работоспособности при частичных отказах клиентских узлов.

Заключение

В рамках данной работы была поставлена цель: предложить и разработать рабочий прототип альтернативного подхода к инференсу больших языковых моделей, основанных на архитектуре Mixture-of-Experts (MoE), удовлетворяющий требованиям динамической балансировки нагрузки и обеспечивающий более высокую отказоустойчивость по сравнению с оригинальным подходом, основанным на коллективах [3; 14].

Основная гипотеза заключалась в том, что замена коллективных All-to-All коммуникаций на уровне экспертного слоя в MoE моделях на клиент-серверное взаимодействие между основной частью модели и экспертными блоками позволяет удовлетворить поставленным требованиям. В качестве ориентира для коллективного подхода в работе рассматривается библиотека DeepEP [2].

В процессе проверки гипотезы и проведения сопутствующих исследований был предложен и реализован прототип клиент-серверной архитектуры с асинхронной диспетчеризацией экспертов и механизмом передачи памяти между видеокартами. Система включает компоненты мониторинга состояния узлов, динамической балансировки нагрузки и обнаружения отказов в реальном времени.

Поскольку основная проблема выбранного подхода заключается в потенциальном снижении общей производительности, для ее решения были предложены и реализованы следующие оптимизации: уменьшение размера пакетов данных, которые передаются по сети, эффективное использование шины памяти за счет разбиения передачи данных на части, использование выравнивания экспертных активаций для более эффективного использования кэша GPU-операций, а также применение grouped GeMM для уменьшения количества точек синхронизации и повышения эффективности работы кэша GPU.

В результате проделанной работы спроектирован и разработан прототип экспертного слоя на основе клиент-серверной архитектуры для отказоустойчивого инференса MoE моделей, который обеспечивает возможность перераспределения нагрузки между экспертами и плавную деградацию производительности при отказах узлов, но демонстрирует производительность ниже, чем современные решения, основанные на коллективах. Система способна продолжать работу при выходе из строя отдельных экспертных блоков, в то время как решения, основанные на коллективных операциях, полностью останавливают инференс модели при возникновении отказа на любом из слоев.

В дальнейшей перспективе необходимы оптимизация асинхронного взаимодействия основной части модели и удалённых экспертных блоков, а также интеграция предложенного подхода с существующими системами инференса больших языковых моделей, такими как vLLM, SGLang и TensorRT-LLM [28—30]. Кроме того, представляет интерес разработка автоматических механизмов адаптации конфигурации системы под изменяющиеся паттерны нагрузки и создание библиотеки, реализующей предло-

женные техники для их широкого применения в индустрии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale / R. Y. Aminabadi [и др.] // SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. — 2022.
2. *DeepSeek-AI*. DeepEP: An Efficient Expert-Parallel Communication Library. — 2026. — URL: <https://github.com/deepseek-ai/DeepEP> (дата обр. 17.05.2026).
3. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer / N. Shazeer [и др.] // arXiv preprint arXiv:1701.06538. — 2017.
4. Scaling Laws for Neural Language Models / J. Kaplan [и др.] // arXiv preprint arXiv:2001.08361. — 2020.
5. Training Compute-Optimal Large Language Models / J. Hoffmann [и др.] // arXiv preprint arXiv:2203.15556. — 2022.
6. Attention Is All You Need / A. Vaswani [и др.] // Advances in Neural Information Processing Systems. — 2017.
7. Language Models are Few-Shot Learners / T. B. Brown [и др.] // Advances in Neural Information Processing Systems. — 2020.
8. *DeepSeek-AI*. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models // arXiv preprint arXiv:2512.02556. — 2025.
9. *Fedus W., Zoph B., Shazeer N.* Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity // arXiv preprint arXiv:2101.03961. — 2021.
10. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts / N. Du [и др.] // arXiv preprint arXiv:2112.06905. — 2021.
11. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism / M. Shoeybi [и др.] // arXiv preprint arXiv:1909.08053. — 2019.
12. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM / D. Narayanan [и др.] // Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. — 2021.
13. Towards Low-bit Communication for Tensor Parallel LLM Inference / H. Dong [и др.] // arXiv preprint arXiv:2411.07942. — 2024.
14. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale / S. Rajbhandari [и др.] // Proceedings of the 39th International Conference on Machine Learning. — 2022.
15. *NVIDIA*. NVIDIA Collective Communication Library Documentation. — 2026. — URL: <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/index.html> (дата обр. 17.05.2026).

16. *NVIDIA*. NVIDIA OpenSHMEM Library Documentation. — 2026. — URL: <https://docs.nvidia.com/nvshmem/api/> (дата обр. 17.05.2026).
17. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding / D. Lepikhin [и др.] // arXiv preprint arXiv:2006.16668. — 2020.
18. *DeepSeek-AI*. EPLB: Expert Parallelism Load Balancer. — 2026. — URL: <https://github.com/deepseek-ai/EPLB> (дата обр. 18.05.2026).
19. *NVIDIA*. NVLink and NVSwitch: Fastest HPC Data Center Platform. — 2026. — URL: <https://www.nvidia.com/object/nvlink.html> (дата обр. 17.05.2026).
20. *OpenUCX*. UCX Main Features. — 2026. — URL: https://openucx.readthedocs.io/en/master/ucx_features.html (дата обр. 17.05.2026).
21. *AI-Dynamo*. NVIDIA Inference Xfer Library. — 2026. — URL: <https://github.com/ai-dynamo/nixl> (дата обр. 17.05.2026).
22. *NVIDIA*. NVIDIA Enters Production With Dynamo, the Broadly Adopted Inference Operating System for AI Factories. — 2026. — URL: <https://nvidianews.nvidia.com/news/nvidia-enters-production-with-dynamo-the-broadly-adopted-inference-operating-system-for-ai-factories> (дата обр. 17.05.2026).
23. *NVIDIA*. NVIDIA Dynamo Documentation: Disaggregated Serving. — 2026. — URL: <https://docs.dynamo.nvidia.com/dynamo/design-docs/disaggregated-serving> (дата обр. 18.05.2026).
24. *MoonCake*. MoonCake Supported Communication Protocols. — 2026. — URL: https://kvcache-ai.github.io/Mooncake/getting_started/supported-protocols.html (дата обр. 17.05.2026).
25. *MoonCake*. MoonCake Transfer Engine. — 2026. — URL: <https://kvcache-ai.github.io/Mooncake/design/transfer-engine/index.html> (дата обр. 17.05.2026).
26. *NVIDIA*. GPUDirect RDMA Documentation. — 2026. — URL: <https://docs.nvidia.com/cuda/gpudirect-rdma/> (дата обр. 17.05.2026).
27. *Qwen Team*. Qwen3 Technical Report // arXiv preprint arXiv:2505.09388. — 2025.
28. *vLLM Project*. vLLM: A High-Throughput and Memory-Efficient Inference and Serving Engine for LLMs. — 2026. — URL: <https://github.com/vllm-project/vllm> (дата обр. 18.05.2026).
29. *SGLang Project*. SGLang: High-Performance Serving Framework for LLMs and VLMs. — 2026. — URL: <https://www.sglang.io/> (дата обр. 18.05.2026).
30. *NVIDIA*. TensorRT-LLM. — 2026. — URL: <https://github.com/NVIDIA/TensorRT-LLM> (дата обр. 18.05.2026).